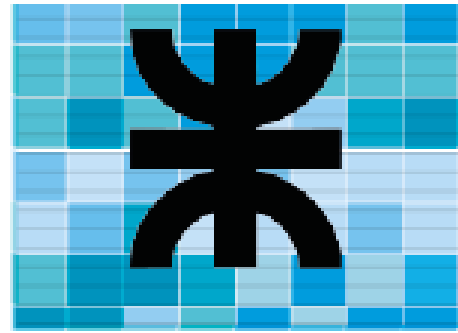
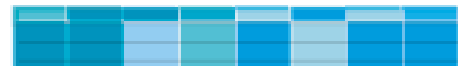




UTN

*Universidad Tecnológica Nacional
Ingeniería en Sistemas de Información*



Tipos de Dato- Tipos de Dato Abstracto y Operadores Relacionales

Ing. María Inés Seguenzia

Repaso de Conceptos

- Algoritmo- Programa- Lenguaje de programación
- Estructura de Datos
 - Variable – **Constante** (práctica 1)
 - Arreglos (práctica 2 y 3)
 - Registros
 - Archivos
- Estructuras de Control
 - Sentencias Entrada, Salida y Asignación ←
 - Decisión Simple **IF**
 - **Decisión Múltiple MATCH Case**
 - Estructuras de Repetición
 - Para ó **FOR**
 - Mientras **WHILE**
 - Repetir Hasta**

Vamos a
ENCORSETAR a
PYTHON para que
se ajuste a nuestro
Paradigma de
Programación
Que es la
**PROGRAMACIÓN
ESTRUCTURADA**

Tipos de Datos: definición

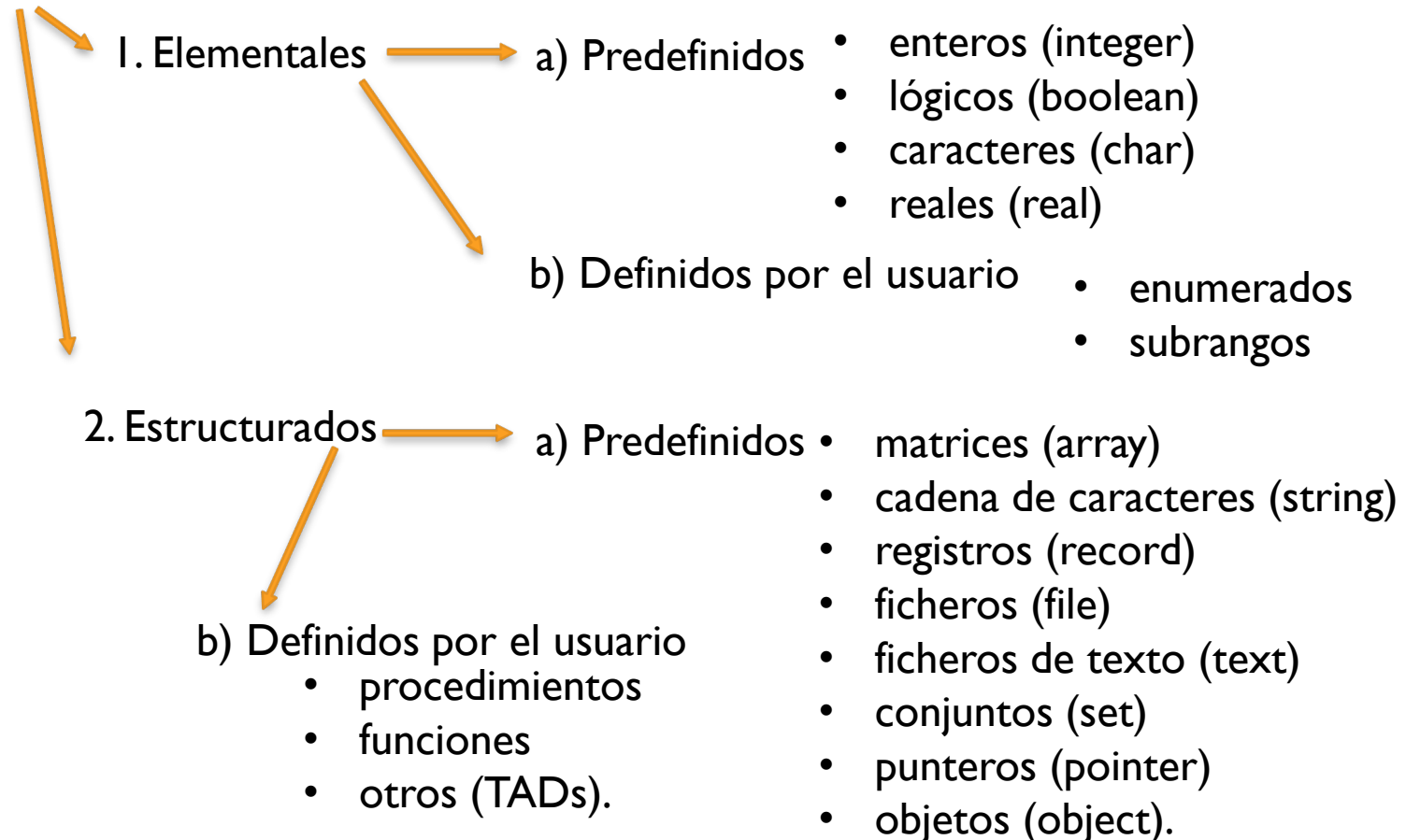
- Un **tipo de datos** es la propiedad de un valor que determina su dominio (**qué valores puede tomar**), qué **operaciones** se le pueden aplicar y cómo es representado internamente por el computador.
- Todos los valores que aparecen en un programa pertenecen a un tipo.

Clasificación de tipos de datos según distintos criterios:

En Python el tipo de dato **Char** es representado por un **string de un elemento**

- Según si modifican su valor en ejecución:
 - constantes (const)
 - variables (var).

- Según su composición:



Clasificación de tipos de datos

según distintos criterios:

- Según la naturaleza de los valores:
- numéricos
 - entero
 - real
 - caracteres
 - char
 - string

– Según su almacenamiento en memoria

(¿Cómo? y ¿Dónde?):

- estáticos (simples y estructurados)
- dinámicos (archivos, variables dinámicas)

– Según su ordenación:

ordinales

(Predecesor-Sucesor)

- entero
- boolean
- char
- enumerados
- subrango

*Ej. Escolaridad:
primaria,
secundaria,
terciaria,
universitaria*

escalares

*(que contiene un
valor a la vez)*

- entero
- boolean
- char
- real

Tipo de Dato Abstracto

- Una abstracción es la simplificación de un objeto o de un proceso de la realidad en la que sólo se consideran los aspectos más relevantes.



- En programación, la abstracción permite considerar la resolución de un problema sin tener en cuenta los detalles. Da sencillez al algoritmo.
Concepto de Modularización
- Surgen para facilitar el trabajo con tipos de datos sin preocuparnos por la implementación.
- Un **Tipo de dato** abstracto (TDA) es un conjunto de **datos** al cual se le asocian **operaciones**.

¿Los tipos de datos simples ó elementales son TDA?

Definición de los tipos de datos predefinidos como TDA:

- El TDA **entero** tiene como posibles valores de los datos el conjunto $\{-1, -2, \dots, \infty\} \cup \{0, 1, 2, \dots, \infty\}$, y como operaciones **la suma, la resta, la multiplicación y la división entera**.
- El TDA **real** tiene como tipo el conjunto de los puntos de la recta real **(Z+Q+I)** y como operaciones **la suma, la resta, la multiplicación y la división**.
- El TDA **Booleano** tiene como conjunto de valores **{True, False}** y como operaciones las definidas por el álgebra de Boole **(AND, OR, NOT)**.
- El TDA **carácter** tiene como tipo el conjunto de caracteres definido por un **alfabeto** dado y como operaciones todos los operadores relacionales **(=, <>, ≥, ≤)**.

Expresiones lógicas: Operadores Relacionales y Conectores Lógicos

Una **expresión** es una fórmula compuesta por variables y constantes (**operandos**) y por símbolos que indican la aplicación de una acción (**operadores**).

Expresión Aritmética: Ejemplo: $a=12$ $b=8$ $c=a+b$

- Su resultado es un número.
- Utilizan operadores aritméticos: suma, resta, producto, etc.

Expresión Lógica: Ejemplo: $(med > 1.63)$ **and** $(med < 1.67)$

- Su resultado es un valor de verdad (True o False).
- Utilizan operadores relacionales y lógicos.

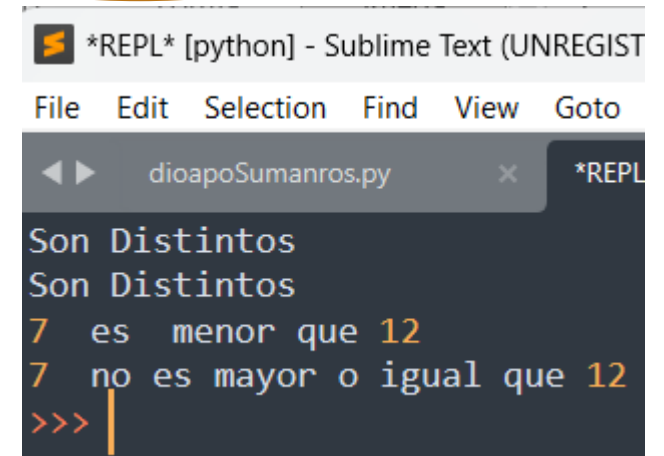
Operadores relacionales o de Comparación

Opera dor	Significado	ejemplo	Observaciones
==	igual que	$a == b$	retorna <i>True</i> si <i>a</i> es igual que <i>b</i> , ó <i>False</i> en caso contrario
!=	distinto de	$a != b$	retorna <i>True</i> si <i>a</i> es distinto de <i>b</i> , ó <i>False</i> en caso contrario
>	mayor que	$a > b$	retorna <i>True</i> si <i>a</i> es mayor que <i>b</i> , ó <i>False</i> en caso contrario
<	menor que	$a < b$	retorna <i>True</i> si <i>a</i> es menor que <i>b</i> , ó <i>False</i> en caso contrario
>=	mayor o igual que	$a >= b$	retorna <i>True</i> si <i>a</i> es mayor o igual que <i>b</i> , ó <i>False</i> en caso contrario
<=	menor o igual que	$a <= b$	retorna <i>True</i> si <i>a</i> es menor o igual que <i>b</i> , ó <i>False</i> en caso contrario

Comparación de números

```
n1 = 7
n2 = 12
if n1 == n2:
    print('son Iguales')
else:
    print('Son Distintos')
if n1 != n2:
    print('Son Distintos')
else:
    print('Son Iguales')
if n1 < n2:
    print(n1, ' es menor que', n2)
else:
    print(n1, ' no es menor que', n2)
if n1 >= n2:
    print(n1, 'es mayor o igual que', n2)
else:
    print(n1, ' no es mayor o igual que', n2)
```

**Observar son
IF SECUENCIALES
(uno a continuación
del otro)**



```
*REPL* [python] - Sublime Text (UNREGIST
File Edit Selection Find View Goto
dioapoSumanros.py x *REPL
Son Distintos
Son Distintos
7 es menor que 12
7 no es mayor o igual que 12
>>> |
```

Comparación de cadenas

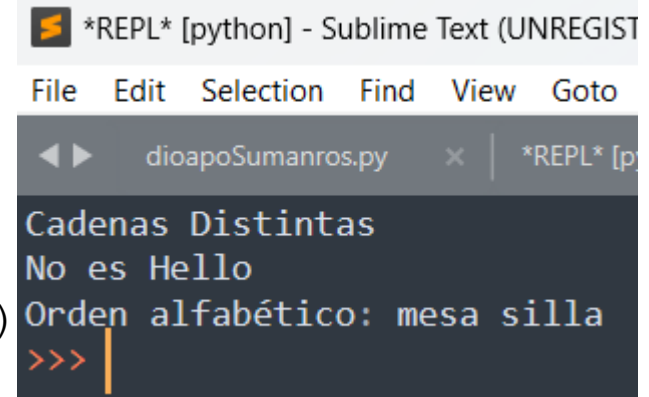
```
cad1 = 'Hola'
cad2 = 'Mundo'
```

```
if cad1 == cad2:
    print('Iguales')
else:
    print('Cadenas Distintas')
```

```
if cad1 != 'Hello':
    print('No es Hello')
else:
    print('Es Hello...')
```

```
cad1 = 'mesa'
cad2 = 'silla'
```

```
if cad1 < cad2:
    print('Orden alfabético:', cad1, cad2)
else:
    print('Orden alfabético:', cad2, cad1)
```

A screenshot of a Sublime Text REPL window titled '*REPL* [python] - Sublime Text (UNREGISTERED)'. The window has a menu bar with 'File', 'Edit', 'Selection', 'Find', 'View', and 'Goto'. Below the menu bar, there's a tab labeled 'dioapoSumanros.py' and another labeled '*REPL* [python]'. The main area shows the output of a Python script: 'Cadenas Distintas', 'No es Hello', and 'Orden alfabético: mesa silla'. The prompt '>>>' is visible at the bottom left of the output area.

```
*REPL* [python] - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto
dioapoSumanros.py × *REPL* [python]
Cadenas Distintas
No es Hello
Orden alfabético: mesa silla
>>> |
```

Operadores Lógicos

- . Un conector lógico u operador booleano es un operador que permite encadenar la comprobación de dos o más expresiones lógicas y obtener un resultado único.
- . La expresión lógica encadenada por un conector lógico se designa como proposición lógica.

Operador	Significado	Ejemplo
and	conjunción lógica (y)	<code>a == b and y != x</code>
or	disyunción lógica (o)	<code>n == 1 or n == 2</code>
not	negación lógica (no)	<code>not x > 7</code>

Tablas de Verdad

Sean p y q dos proposiciones lógicas:

Tabla de verdad del conector <i>and</i>		
p	q	p <i>and</i> q
True	True	True
True	False	False
False	True	False
False	False	False

Tabla de verdad del conector <i>or</i>		
p	q	p <i>or</i> q
True	True	True
True	False	True
False	True	True
False	False	False

Tablas de Verdad

Ejemplo:

and	<pre>if sueldo > 150000 and antigüedad >= 10: print('Crédito concedido') else: print('Crédito rechazado')</pre>
or	<pre>if angulo1 == 90 or angulo2 == 90 or angulo3 == 90 print('es un triángulo rectángulo') else: print('No es un triángulo rectángulo')</pre>

Deben cumplirse
ambas
condiciones para
que sea verdadero

Con que una de las
condiciones sea
verdadera, la salida
será verdadera

Tablas de Verdad

Tabla de verdad del conector *not*

p	<i>not</i> p
True	False
False	True

```
if not edad < 18:  
    print('Accede al permiso')  
else:  
    print ('No  
accede al permiso')
```

```
if edad >= 18:  
    print('Accede al  
permiso')  
else:  
    print ('No  
accede al permiso')
```

Precedencia de operadores matemáticos

- Primero se aplican los operadores de *multiplicación* (*), *división* (//, /), *resto* (%) y *potencia* (**).
- Luego, los operadores de *suma* (+) y *resta* (-).
- Si dos operadores tienen la misma precedencia, se aplica primero el que esté más a la izquierda.
- Cualquier expresión encerrada entre paréntesis se ejecutará primero. Los paréntesis se usan para cambiar la precedencia de ejecución de algún operador.

Precedencia de operadores

matemáticos > relacionales > conectores

+ - * /

< > >= <=
== !=

not and or

Los operadores relacionales tienen la misma precedencia entre ellos y se aplican de izquierda a derecha.

- `if a > b > c:`
- `if a > b and b > c:`

En los conectores lógicos la precedencia es:

- `precedencia(not) > precedencia(and) > precedencia(or)`

Precedencia de operadores

Conector	Ejemplo	Salida
and	if $a > b$ and $a > c$:	Si la primera proposición es <i>False</i> ($a > b$ en este caso), la segunda ($a > c$) <i>no se chequea</i> y la salida también es <i>False</i> .
or	if $n < 0$ or $n > 9$:	Si la primera proposición es <i>True</i> ($n < 0$ en este caso), la segunda ($n > 9$) <i>no se chequea</i> y la salida también es <i>True</i> .

Generación de números aleatorios

- *Los números aleatorios (o números random) son números seleccionados al azar (o al menos, en una forma que no sea predecible por el usuario ni por el mismo programador) .*
- La obtención del número aleatorio se basa en algún algoritmo que partiendo de un valor inicial (*valor semilla* o *seed* del generador) es capaz de generar una secuencia de aspecto aleatorio (*generación de números pseudo-aleatorios*).

Generación de números aleatorios

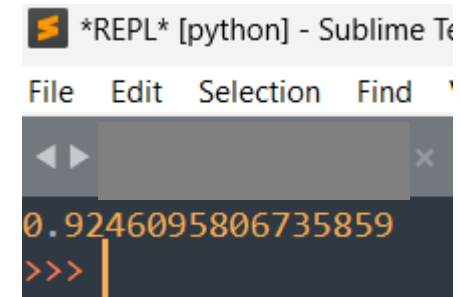
- Python posee un *módulo* (o una *librería*) llamado *random*, que contiene las definiciones y funciones necesarias para poder acceder a la gestión de números *pseudo-aleatorios*.
- Para usar el contenido del módulo se usa la instrucción *import random* al inicio del script en donde se requiera.
- Para acceder a las funciones se coloca el prefijo "*random.*" antes del nombre de la función.

Generación de números aleatorios

random(): calcula y retorna un *valor de probabilidad*: un número en coma flotante pseudo-aleatorio dentro del intervalo $[0, 1)$ (incluye al cero como posible salida, pero no al 1).

Ejemplo:

```
import random
x = random.random()
print(x)
```



The screenshot shows a terminal window titled '*REPL* [python] - Sublime Te'. It has a menu bar with 'File', 'Edit', 'Selection', and 'Find'. Below the menu bar is a scrollbar and a close button. The main area shows the output of a Python command: '0.9246095806735859'. Below the output is a prompt '>>>' with a cursor.

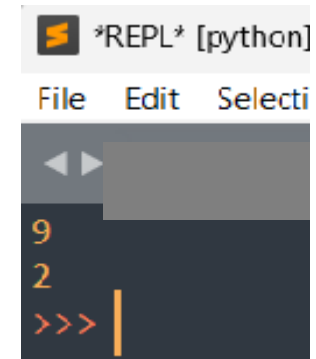
Generación de números aleatorios

- ***randrange(a, b)***: que obtiene y retorna un número entero pseudo-aleatorio pero comprendido en el intervalo $[a, b-1]$ (incluye al valor a como posible salida, pero no al b) .
- ***randint(a, b)***: que permite hacer lo mismo que *random.randrange(a, b)*, pero de forma tal que el número retornado estará en el intervalo $[a, b]$ (incluirá a b como posible salida) .

```
import random
```

```
# 2 <= número aleatorio entero < 10  
n1 = random.randrange(2, 10)  
print(n1)
```

```
# 2 <= número aleatorio entero <= 10  
n2 = random.randint(2, 10)  
print(n2)
```



Generación de números aleatorios

- ***choice(sec)***: acepta una *secuencia* (una tupla, una cadena, una lista, etc.) y retorna un elemento cualquiera de esa secuencia, elegido al azar.

Ejemplo:

```
import random

sec1 = 2, 10, 7, 9, 3, 4
r1 = random.choice(sec1)
print(r1)

sec2 = 'ABCDEFGHI'
r2 = random.choice(sec2)
print(r2)
```

